上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Mutation Test

These slides are copyrighted. They are for use with the **Foundations of Software Testing** book by Aditya Mathur. Please use the slides but do not remove the copyright notice.

LEARNING OBJECTIVES

- What is test adequacy? What is test enhancement? When to measure test adequacy and how to use it to enhance tests?
- What is program mutation?
- Competent programmer hypothesis and the coupling effect.
- Strengths and limitations of test adequacy based on program mutation.
- Mutation operators
- Tools for mutation testing

Copyright© 2008 Dorling Kindersley India Pvt Ltd

What is adequacy?

- Consider a program P written to meet a set R of functional requirements. We notate such a P and R as (P, R). Let R contain n requirements labeled $R_1, R_2, \ldots, R_n$.

- Suppose now that a set T containing k tests has been constructed to test P to determine whether or not it meets all the requirements in R . Also, P has been executed against each test in T and has produced correct behavior.

- We now ask: Is T good enough? This question can be stated differently as: Has P been tested thoroughly?, or as: Is T adequate?

Copyright© 2008 Dorling Kindersley India Pvt Ltd

What is program mutation?

- Suppose that program P has been tested against a test set T and P has not failed on any test case in T. Now suppose we do the following:



What behavior do you expect from P′ against tests in T?

Copyright© 2008 Dorling Kindersley India Pvt Ltd

What is program mutation? [2]

- P' is known as a mutant of P.
- There might be a test t in T such that P(t)≠P'(t). In this case we say that t distinguishes P' from P. Or, that t has killed P'.
- There might be not be any test t in T such that P(t)≠P'(t). In this case we say that T is unable to distinguish P and P'. Hence P' is considered live in the test process.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

What is program mutation? [3]

- If there does not exist any test case t in the input domain of P that distinguishes P from P' then P' is said to be equivalent to P.
- If P' is not equivalent to P but no test in T is able to distinguish it from P then T is considered inadequate.
- A non-equivalent and live mutant offers the tester an opportunity to generate a new test case and hence enhance T.

We will refer to program mutation as mutation.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Test adequacy using mutation [1]

- Given a test set T for program P that must meet requirements R, a test adequacy assessment procedure proceeds as follows.

- Step 1: Create a set M of mutants of P. Let M={ $M_1 \ldots M_k$}. Note that we have k mutants.

- Step 2: For each mutant M_i find if there exists a t in T such that $M_i(t) \neq P(t)$. If such a t exists then M_i is considered killed and removed from further consideration.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Test adequacy using mutation [2]

- Step 3: At the end of Step 2 suppose that $k_1 \leq k$ mutants have been killed and $(k-k_1)$ mutants are live.

  Case 1: $(k-k_1)=0$: T is adequate with respect to mutation.

  Case 2: $(k-k_1)>0$ then we compute the mutation score (MS) as follows:

  $$MS=k_1/(k-e)$$

  Where e is the number of equivalent mutants. Note: $e \leq (k-k_1)$.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Test enhancement using mutation

- One has the opportunity to enhance a test set T after having assessed its adequacy.
- Step 1: If the mutation score (MS) is 1, then some other technique, or a different set of mutants, needs to be used to help enhance T.
- Step 2: If the mutation score (MS) is less than 1, then there exist live mutants that are not equivalent to P. Each live mutant needs to be distinguished from P.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Test enhancement using mutation [2]

- Step 3: Hence a new test t is designed with the objective of distinguishing at least one of the live mutants; let us say this is mutant m.
- Step 4: If t does not distinguish m then another test t needs to be designed to distinguish m. Suppose that t does distinguish m.
- Step 5: It is also possible that t also distinguishes other live mutants.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Test enhancement using mutation [3]

- Step 6: One now adds t to T and re-computes the mutation score (MS).

- Repeat the enhancement process from Step 1.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Error detection using mutation

- As with any test enhancement technique, there is no guarantee that tests derived to distinguish live mutants will reveal a yet undiscovered error in P. Nevertheless, empirical studies have found to be the most powerful of all formal test enhancement techniques.
- The next simple example illustrates how test enhancement using mutation detects errors.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Error detection using mutation [2]

- Consider the following function foo that is required to return the sum of two integers x and y. Clearly foo is incorrect.

```
int foo(int x, y){
return (x-y);   <------   This should be return (x+y)
}
```

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Error detection using mutation [3]

- Now suppose that foo has been tested using a test set T that contains two tests:

$$T=\{ t1: \langle x=1, y=0\rangle, t2: \langle x=-1, y=0\rangle\}$$

- First note that foo behaves perfectly fine on each test in, i.e. foo returns the expected value for each test case in T. Also, T is adequate with respect to all control and data flow based test adequacy criteria.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Error detection using mutation [4]

Let us evaluate the adequacy of T using mutation. Suppose that the following three mutants are generated from foo.

M1:
```
int foo(int x, y){
return (x+y);
}
```

M2:
```
int foo(int x, y){
return (x-0);
}
```

M3:
```
int foo(int x, y){
return (0+y);
}
```

- Note that M1 is obtained by replacing the - operator by a + operator, M2 by replacing y by 0, and M3 by replacing x by 0.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Error detection using mutation [4]

Next we execute each mutant against tests in T until the mutant is distinguished or we have exhausted all tests. Here is what we get.

T={ t1: <x=1, y=0>, t2: <x=-1, y=0>}

Test (t)	foo(t)	M1(t)	M2(t)	M3(t)
t1	1	1	1	0
t2	-1	-1	-1	0
		Live	Live	Killed

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Error detection using mutation [5]

After executing all three mutants we find that two are live and one is distinguished. Computation of mutation score requires us to determine of any of the live mutants is equivalent.

In class exercise: Determine whether or not the two live mutants are equivalent to foo and compute the mutation score of T.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Error detection using mutation [6]

Let us examine the following two live mutants.

M1:
```
int foo(int x, y){
return (x+y);
}
```

M2:
```
int foo(int x, y){
return (x-0);
}
```

Let us focus on M1. A test that distinguishes M1 from foo must satisfy the following condition:

$x-y \neq x+y$ implies $y \neq 0$.

Hence we get t3: <x=1, y=1>

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Error detection using mutation [7]

Executing foo on t3 gives us foo(t3)=0. However, according to the requirements we must get foo(t3)=2. Thus t3 distinguishes M1 from foo and also reveals the error.

M1:
```
int foo(int x, y){
return (x+y);
}
```

M2:
```
int foo(int x, y){
return (x-0);
}
```

In class exercise: (a) Will any test that distinguishes also reveal the error? (b) Will any test that distinguishes M2 reveal the error?

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Guaranteed error detection

Sometimes there exists a mutant P' of program P such that any test t that distinguishes P' from P also causes P to fail. More formally:

> Let P' be a mutant of P and t a test in the input domain of P. We say that P' is an error revealing mutant if the following condition holds for any t.

> $P'(t) \neq P(t)$ and $P(t) \neq R(t)$, where R(t) is the expected response of P based on its requirements.

Is M1 in the previous example an error revealing mutant? What about M2?

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Distinguishing a mutant

A test case t that distinguishes a mutant m from its parent program P program must satisfy the following three conditions:

- Condition 1:可达性：测试用例*t* 执行时需要覆盖到变异语句*s* 。因为变异体*m* 内其他语句与*P* 相同，如果测试用例*t* 不能执行*m* 中的*s* 语句，则*t* 在*P* 和*m*上的运行结果必然一致，即*t* 不能检测到变异体*m* 。

- Condition 2: .传染性：测试用例*t* 在执行完*P* 和*m* 的语句*s* 后程序状态不一致。

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Distinguishing a mutant [2]

- Condition 3:传播性：执行完语句*s* 后的不一致的程序状态将导致程序*P* 和*m* 的输出不一致。

Exercise: Show that in the previous example both t1 and t2 satisfy the above three conditions for M3.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Equivalent mutants

- The problem of deciding whether or not a mutant is equivalent to its parent program is undecidable. Hence there is no way to fully automate the detection of equivalent mutants.
- The number of equivalent mutants can vary from one program to another. However, empirical studies have shown that one can expect about 5% of the generated mutants to the equivalent to the parent program.
- Identifying equivalent mutants is generally a manual and often time consuming--as well as frustrating--process.

Table 3 A typical equivalent mutant

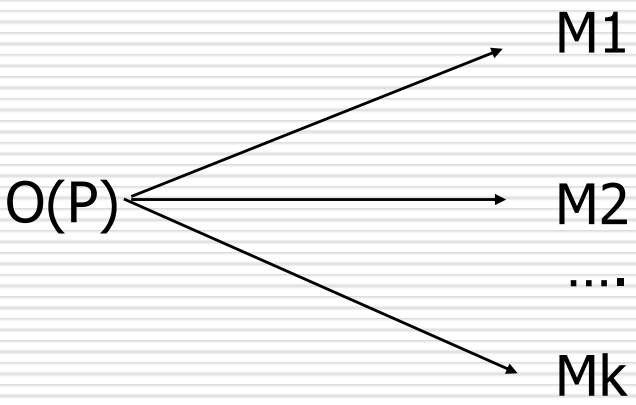
表3 一个典型的等价变异体

程序 p	变异体 p'
for (int $i=0; i<10; i++$){	for (int $i=0; i!=10; i++$){
$sum+=a[i];$	$sum+=a[i];$
}	}

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutant operators

- A mutant operator O is a function that maps the program under test to a set of k (zero or more) mutants of P.



Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutant operators [2]

- A mutant operator creates mutants by making simple changes in the program under test.

- For example, the “variable replacement” mutant operator replaces a variable name by another variable declared in the program. An “relational operator replacement” mutant operator replaces relational operator with another relational operator.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutant operators: Examples

Mutant operator	In P	In mutant
Variable replacement	z=x*y+1;	x=x*y+1; z=x*x+1;
Relational operator replacement	if (x<y)	if(x>y) if(x<=y)
Off-by-1	z=x*y+1;	z=x*(y+1)+1; z=(x+1)*y+1;
Replacement by 0	z=x*y+1;	z=0*y+1; z=0;
Arithmetic operator replacement	z=x*y+1;	z=x*y-1; z=x+y-1;

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutants: First order and higher order

- A mutant obtained by making exactly "one change" is considered first order.

- A mutant obtained by making two changes is a second order mutant. Similarly higher order mutants can be defined. For example, a second order mutant of z=x+y; is x=z+y; where the variable replacement operator has been applied twice.

- In practice only first order mutants are generated for two reasons: (a) to lower the cost of testing and (b) most higher order mutants are killed by tests adequate with respect to first order mutants. [See coupling effect later.]

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutant operators: basis

- A mutant operator models a simple mistake that could be made by a programmer
- Several error studies have revealed that programmers--novice and experts--make simple mistakes. For example, instead of using x<y+1 one might use x<y.
- While programmers make "complex mistakes" too, mutant operators model simple mistakes. As we shall see later, the "coupling effect" explains why only simple mistakes are modeled.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutant operators: Goodness

- The design of mutation operators is based on guidelines and experience. It is thus evident that two groups might arrive at a different set of mutation operators for the same programming language. How should we judge whether or not that a set of mutation operators is "good enough?"

- Informal definition:
  - Let S1 and S2 denote two sets of mutation operators for language L. Based on the effectiveness criteria, we say that S1 is superior to S2 if mutants generated using S1 guarantee a larger number of errors detected over a set of erroneous programs.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutant operators: Goodness [2]

- Generally one uses a small set of highly effective mutation operators rather than the complete set of operators.

- Experiments have revealed relatively small sets of mutation operators for C and Fortran. We say that one is using "constrained" or "selective" mutation when one uses this small set of mutation operators.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutant operators: Language dependence

- For each programming language one develops a set of mutant operators.
- Languages differ in their syntax thereby offering opportunities for making mistakes that duffer between two languages. This leads to differences in the set of mutant operators for two languages.
- Mutant operators have been developed for languages such as Fortran, C, Ada, Lisp, and Java. [See the text for a comparison of mutant operators across several languages.]

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Competent programmer hypothesis (CPH)

- CPH states that given a problem statement, a programmer writes a program P that is in the general neighborhood of the set of correct programs.
- An extreme interpretation of CPH is that when asked to write a program to find the account balance, given an account number, a programmer is unlikely to write a program that deposits money into an account. Of course, while such a situation is unlikely to arise, a devious programmer might certainly write such a program.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Competent programmer hypothesis (CPH) [2]

- A more reasonable interpretation of the CPH is that the program written to satisfy a set of requirements will be a few mutants away from a correct program.
- The CPH assumes that the programmer knows of an algorithm to solve the problem at hand, and if not, will find one prior to writing the program.
- It is thus safe to assume that when asked to write a program to sort a list of numbers, a competent programs knows of, and makes use of, at least one sorting algorithm. Mistakes will lead to a program that can be corrected by applying one or more first order mutations.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Coupling effect

- The coupling effect has been paraphrased by DeMillo, Lipton, and Sayward as follows: “Test data that distinguishes all programs differing from a correct one by only simple errors is so sensitive that it also implicitly distinguishes more complex errors”
- Stated alternately, again in the words of DeMillo, Lipton and Sayward “..seemingly simple tests can be quite sensitive via the coupling effect.”

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Coupling effect [2]

- For some input, a non-equivalent mutant forces a slight perturbation in the state space of the program under test. This perturbation takes place at the point of mutation and has the potential of infecting the entire state of the program.
- It is during an analysis of the behavior of the mutant in relation to that of its parent that one discovers complex faults.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Tools for mutation testing

- As with any other type of test adequacy assessment, mutation based assessment must be done with the help of a tool.

- There are few mutation testing tools available freely. Two such tools are Proteum for C from Professor Maldonado and muJava for Java from Professor Jeff Offutt. We are not aware of any commercially available tool for mutation testing. See the textbook for a more complete listing of mutation tools.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Tools for mutation testing: Features

- A typical tool for mutation testing offers the following features
  - A selectable palette of mutation operators.
  - Management of test set T.
  - Execution of the program under test against T and saving the output for comparison against that of mutants.
  - Generation of mutants.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Tools for mutation testing: Features [2]

- Mutant execution and computation of mutation score using user identified equivalent mutants.
- Incremental mutation testing: i.e. allows the application of a subset of mutation operators to a portion of the program under test.
- Mothra, an advanced mutation tool for Fortran also provided automatic test generation using DeMillo and Offutt's method.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutation and system testing

- Adequacy assessment using mutation is often recommended only for relatively small units, e.g. a class in Java or a small collection of functions in C.
- However, given a good tool, one can use mutation to assess adequacy of system tests.
- The following procedure is recommended to assess the adequacy of system tests.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutation and system testing [2]

- Step 1: Identify a set U of application units that are critical to the safe and secure functioning of the application. Repeat the following steps for each unit in U.
- Step 2: Select a small set of mutation operators. This selection is best guided by the operators defined by Eric Wong or Jeff Offutt. [See book for details.]
- Step 3: Apply the operators to the selected unit.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutation and system testing [3]

- Step 4: Assess the adequacy of T using the mutants so generated. If necessary, enhance T.
- Step 5: Repeat Steps 3 and 4 for the next unit until all units have been considered.
- We have now assessed T, and perhaps enhanced it. Note the use of incremental testing and constrained mutation (i.e. use of a limited set of highly effective mutation operators).

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Mutation and system testing [4]

- Application of mutation, and other advanced test assessment and enhancement techniques, is recommended for applications that must meet stringent availability, security, safety requirements.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Summary

- Mutation testing is the most powerful technique for the assessment and enhancement of tests.
- Mutation, as with any other test assessment technique, must be applied incrementally and with assistance from good tools.
- Identification of equivalent mutants is an undecidable problem--similar the identification of infeasible paths in control or data flow based test assessment.

Copyright© 2008 Dorling Kindersley India Pvt Ltd

Summary [2]

- While mutation testing is often recommended for unit testing, when done carefully and incrementally, it can be used for the assessment of system and other types of tests applied to an entire application.tests.

- Mutation is a highly recommended technique for use in the assurance of quality of highly available, secure, and safe systems.

Copyright© 2008 Dorling Kindersley India Pvt Ltd